ALGORITMOS PARA NÃO PROGRAMADORES

Uma Introdução Prática

Um material baseado na "Cartilha do Octa"

Versão 1.0 — 1ª Edição Revisada (2026)

Repositório de Atualizações: `https://github.com/X5GuilhermeBR/livros`

Apresentação

Este material foi concebido especificamente para alunos de pós-graduação e mestrado vindos de diversas áreas do conhecimento. O objetivo é desmistificar a lógica de programação sem perder a clareza e o rigor conceitual.

Criado por: Guilherme José

Rio de Janeiro – RJ

Licença de Uso e Distribuição
Este material está licenciado sob a **CC BY-NC 4.0** (`https://creativecommons.org/licenses/by-nc/4.0/`).
Permissão Adicional: Instituições de ensino, públicas ou privadas (incluindo escolas e cursos pagos), podem usar, copiar, distribuir e adaptar este material em suas atividades didáticas. É expressamente proibida a venda deste material, ou de versões adaptadas dele, como produto final.
Os exemplos de código contidos nesta obra estão sob a **Licença MIT**.
Código-fonte e atualizações disponíveis em: `https://github.com/X5GuilhermeBR/livros`

Contents

1 Capítulo 1: O que é Algoritmo? 2

1.1 Definição 2

2 Capítulo 2: Informação e suas Representações 3

2.1 Tipos Primitivos de Dados 3

2.2 Constantes, Variáveis e Identificadores 3

2.3 Declaração de Variáveis e Alocação em Memória 4

2.4 Comando de Atribuição (←) 4

3 Capítulo 3: Operadores 6

3.1 Operadores Aritméticos 6

3.2 Operadores Relacionais 6

3.3 Operadores Lógicos 6

4 Capítulo 4: Comandos Básicos 9

4.1 Comando de Entrada (`Leia`) 9

4.2 Comando de Saída (`Escreva/Printf`) 9

4.3 Exemplo Prático Completo 9

5 Capítulo 5: Estruturas de Seleção e Repetição em C 11

5.1 1. Estruturas de Seleção (Condicionais) 11

5.2 2. Estruturas de Repetição (Laços / Loops) 12

6 Capítulo 6: Exercícios de Fixação 16

7 Capítulo 7: Funções e Procedimentos 19

7.1 1. O que é um Procedimento? (`void`) 19

7.2 2. O que é uma Função? 19

7.3 3. Exercícios Propostos 21

1 Capítulo 1: O que é Algoritmo?

A lógica atua como a correção do pensamento, permitindo-nos formular raciocínios coerentes e estruturados. No contexto da programação, traduzimos esse raciocínio lógico em **algoritmos**.

1.1 Definição

O que é um Algoritmo?

Um **algoritmo** é uma **sequência de passos**, **logicamente encadeados**, com um **objetivo bem definido**.

Vamos analisar cada parte dessa definição utilizando um exemplo cotidiano:

Exemplo Prático: Troca de uma Lâmpada

1. Pegue uma escada.
2. Posicione-a sob a lâmpada.
3. Pegue uma lâmpada nova.
4. Suba na escada.
5. Retire a lâmpada queimada.
6. Coloque a lâmpada nova.

Desmistificando os três pilares do conceito:

- **Sequência de passos:** Cada item numerado representa um passo claro e individual na execução da tarefa.
- **Logicamente encadeados:** A ordem das ações importa! Se invertêssemos o **passo 4** com o **passo 5** (retirar a lâmpada antes de subir na escada), o processo falharia. Os passos possuem dependência lógica entre si.
- **Objetivo bem definido:** É a meta do algoritmo. Neste exemplo, o objetivo final é garantir que a lâmpada volte a acender ao fim da execução.

2 Capítulo 2: Informação e suas Representações

A **informação** é a matéria-prima do processamento computacional. Os computadores existem devido à sua capacidade incomparável de armazenar, manipular e organizar grandes volumes de informação com altíssima velocidade e precisão.

2.1 Tipos Primitivos de Dados

Na programação, classificamos as informações em categorias fundamentais chamadas Tipos Primitivos. Existem quatro tipos principais:

1. **Inteiro/Int (Z):** Representa o conjunto numérico dos números inteiros sem casa decimal.

   Exemplo: `Ele tem 17 anos.` → Valor: **17**

2. **Real/Float (R):** Representa números fracionários ou com ponto flutuante (casas decimais).

   Exemplo: `O valor de` π `é 3.1415.` → Valor: **3.1415**

3. **Caractere/Varchar:** Representa dados alfanuméricos (combinação de letras, números e símbolos).

   Exemplo: `"O maior clube do Brasil é o Fluminense!"`

4. **Lógico/Bool (Booleano):** Informações que só podem assumir dois estados possíveis: **Verdadeiro** (`True`) ou **Falso** (`False`).

   Exemplo: `Estado da lâmpada: Acesa ou Apagada.`

2.2 Constantes, Variáveis e Identificadores

Para manipular esses dados dentro de um algoritmo, utilizamos **Constantes** e **Variáveis**:

- **Constante:** Informação cujo valor permanece inalterado ao longo de toda a execução do programa.

  Exemplo: `const PI = 3.1415;`

- **Variável:** Espaço reservado que pode assumir diferentes valores ao longo do tempo.

  Exemplo: `nome` ← `'Zé';` e posteriormente `nome` ← `'Xuxa';`

Regras para Regra de Formação de Identificadores (Nomes de Variáveis)

Ao dar nome a variáveis e programas, devemos seguir rigorosamente estas regras:

1. Deve obrigatoriamente **começar com uma letra**.

2. Pode ser seguido por letras e números.
3. **Não pode conter caracteres especiais** (como `#`, `@`, `$`, acentos, etc.).
4. **Não pode conter espaços em branco**.
5. Pode utilizar o caractere de sublinhado (_ ou *underline*).
6. Deve ser autoexplicativo (possuir relação direta com o dado armazenado).

2.3 Declaração de Variáveis e Alocação em Memória

Cada linguagem possui uma estrutura de escrita específica, chamada **Sintaxe**. Em pseudocódigo, padronizamos a declaração para criar o hábito do rigor formal.

```
Var
<tipo_primitivo> <nome_da_variavel>;
```

Nota: Os sinais de < *e* > *indicam blocos que devem ser substituídos pelos nomes escolhidos pelo programador.*

Exemplo de Declaração

```
Var
varchar nome, endereco, sexo;
int idade, quantAlunos;
float salario, faturamento;
```

O que acontece fisicamente ao declarar uma variável?

O sistema operacional reserva um endereço específico no pente de **Memória RAM** correspondente ao identificador e ao tipo de dado:

nome (Caractere)	**idade** (Inteiro)	**salario** (Real)
[Vazio / Disponível]	[Vazio / Disponível]	[Vazio / Disponível]

2.4 Comando de Atribuição (←)

A atribuição é representada pela seta apontando para a esquerda (←) ou pelo sinal de igual (=). Ela atribui o valor que está do lado **direito** para a variável localizada no lado **esquerdo**.

```
clube = "Fluminense";
x = 2 * x - 1;
```

Ao atribuir um valor, a célula de memória anteriormente vazia passa a guardar a nova informação.

3 Capítulo 3: Operadores

Os operadores são símbolos ou palavras reservadas que nos permitem realizar cálculos, comparações e tomadas de decisão sobre os dados armazenados nas variáveis.

3.1 Operadores Aritméticos

São utilizados para realizar operações matemáticas convencionais e cálculos dentro do algoritmo.

Operador	Descrição	Exemplo / Sintaxe	Resultado
`+`	Soma	`7 + 2`	`9`
`-`	Subtração	`9 - 2`	`7`
`*`	Multiplicação	`2 * 9`	`18`
`/`	Divisão Real	`7 / 2`	`3.5`
`div`	Divisão Inteira (quociente)	`7 div 2`	`3`
`resto` / `mod`	Resto da divisão	`7 resto 2`	`1`
`Pot(b, e)`	Potenciação ($base^{expoente}$)	`Pot(7, 2)`	`49`
`Rad(v, i)`	Radiciação ($\sqrt[indice]{valor}$)	`Rad(9, 2)`	`3`

Table 1: Operadores Aritméticos Fundamentais.

Nota sobre Potenciação e Radiciação

Nas funções operacionais:

- `Pot(base, expoente)`: O primeiro parâmetro é a base e o segundo é o expoente.
- `Rad(valor, indice)`: O primeiro parâmetro é o radicando e o segundo é o índice da raiz.

3.2 Operadores Relacionais

São utilizados para realizar comparações entre dois valores ou expressões. O resultado de qualquer comparação relacional é sempre um valor **Lógico** (`Verdadeiro` ou `Falso`).

3.3 Operadores Lógicos

Os operadores lógicos servem para combinar duas ou mais expressões relacionais.

Operador	Descrição	Exemplo (Considerando $A = 5, B = 4, C = 2$)	Resultado
`>`	Maior que	`A > B` $(5 > 4)$	**Verdadeiro**
`>=`	Maior ou igual a	`A >= B` $(5 \geq 4)$	**Verdadeiro**
`<`	Menor que	`A < C` $(5 < 2)$	**Falso**
`<=`	Menor ou igual a	`B <= C` $(4 \leq 2)$	**Falso**
`=`	Igual a	`A = B` $(5 = 4)$	**Falso**
`<>`	Diferente de	`A <> B` $(5 \neq 4)$	**Verdadeiro**

Table 2: Operadores Relacionais e Seus Resultados Lógicos.

A) Operador Lógico `E` (Conjunção)

Exemplo didático: "Se fizer sol **E** você pagar as minhas despesas, eu vou à praia."
Para que a afirmação seja verdadeira, **ambas** as condições precisam ser satisfeitas.

Sentença A (Sol)	Sentença B (Pagar despesas)	Expressão Lógica	Resultado (Vou à praia?)
Verdadeiro	Verdadeiro	`V E V`	**Verdadeiro**
Verdadeiro	Falso	`V E F`	**Falso**
Falso	Verdadeiro	`F E V`	**Falso**
Falso	Falso	`F E F`	**Falso**

Table 3: Tabela Verdade do Operador Lógico `E`.

B) Operador Lógico `OU` (Disjunção)

Exemplo didático: "Se fizer sol **OU** você pagar as minhas despesas, eu vou à praia."
Basta que **pelo menos uma** das condições seja verdadeira para que o resultado final seja verdadeiro.

Sentença A (Sol)	Sentença B (Pagar despesas)	Expressão Lógica	Resultado (Vou à praia?)
Verdadeiro	Verdadeiro	`V OU V`	**Verdadeiro**
Verdadeiro	Falso	`V OU F`	**Verdadeiro**
Falso	Verdadeiro	`F OU V`	**Verdadeiro**
Falso	Falso	`F OU F`	**Falso**

Table 4: Tabela Verdade do Operador Lógico `OU`.

C) Operador Lógico `NÃO` (Negação)

O operador `NÃO` inverte o valor lógico de uma expressão (se era verdadeiro, torna-se falso, e vice-versa).

Entrada (Sentença A)	Resultado (NÃO A)
Verdadeiro	**Falso**
Falso	**Verdadeiro**

Table 5: Tabela Verdade do Operador de Negação NÃO.

Resumo Prático dos Operadores Lógicos

- **E:** Só dá **Verdadeiro** se **TODOS** forem Verdadeiro.
- **OU:** Só dá **Falso** se **TODOS** forem Falso.
- **NÃO: Inverte** o resultado lógico.

4 Capítulo 4: Comandos Básicos

Os programas de computador precisam interagir com os usuários para receber dados (entradas) e exibir os resultados processados (saídas).

4.1 Comando de Entrada (`Leia`)

Para alimentar o programa com dados, o usuário utiliza dispositivos de entrada (como teclado ou leitor óptico). O comando responsável por capturar essa informação e armazená-la em uma variável é o **`Leia/Scanf`**.

```
scanf(nome_da_variavel);
```

O que o programa faz ao encontrar o comando `Leia/Scanf`?

O programa interrompe a execução, aguarda o usuário digitar um valor e pressionar *Enter*, e em seguida armazena o valor digitado na variável indicada.
Exemplo: `scanf(nome_aluno);` ou `scanf(idade);`

4.2 Comando de Saída (`Escreva/Printf`)

O comando de saída é utilizado para exibir mensagens, resultados ou valores armazenados em variáveis na tela ou impressora.

```
printf("Mensagem em texto"); // Exibe um texto fixo
printf(nome_da_variavel);      // Exibe o conteúdo de uma variável
```

Exemplo de Exibição de Dados e Variáveis

```
varchar nome = "Nina";
printf("Bom dia, %s" &nome);
```

Saída esperada no terminal: `Bom dia, Nina`

4.3 Exemplo Prático Completo

Vamos criar um programa simples em C que lê dois números inteiros fornecidos pelo usuário, calcula a soma entre eles e exibe o resultado final.

Programa em C: Soma de Dois Números

```c
#include <stdio.h>

int main() {
int n1, n2, soma;

printf("Informe o primeiro número:  ");
scanf("%d", &n1);

printf("Informe o segundo número:  ");
scanf("%d", &n2);

soma = n1 + n2;

printf("%d + %d = %d\n", n1, n2, soma);

return 0;
}
```

Análise do Fluxo de Execução

1. **Inclusão de Biblioteca e Declaração:** Incluímos a biblioteca `<stdio.h>` para usar entrada e saída. Reservamos espaço na memória RAM para as variáveis inteiras `n1`, `n2` e `soma`.
2. **Entrada (`scanf`):** Exibimos a mensagem com `printf` e lemos os dois inteiros com `scanf("%d", &variavel)`, usando o operador `&` para indicar o endereço de memória onde o valor digitado será guardado.
3. **Processamento:** Realizamos o cálculo da adição com o operador `+` e guardamos o resultado na variável `soma` através do operador de atribuição (`=`).
4. **Saída (`printf`):** Imprimimos na tela o resultado formatado, onde cada especificador `%d` é substituído, na ordem, pelas variáveis `n1`, `n2` e `soma`.

5 Capítulo 5: Estruturas de Seleção e Repetição em C

Até agora, nossos programas eram executados de forma puramente sequencial (linha por linha). Em linguagem **C**, utilizamos **estruturas de seleção** para tomada de decisões e **estruturas de repetição** para executar blocos de código várias vezes.

5.1 1. Estruturas de Seleção (Condicionais)

As estruturas de seleção permitem escolher qual caminho o programa deve seguir com base no resultado de uma **expressão lógica** (`verdadeiro` ou `falso`).

A) Seleção Simples (`if`)

Executa um bloco de código apenas se a condição for verdadeira. Se for falsa, o bloco entre chaves { } é ignorado.

```
if (condicao) {
/* comandos executados se for verdadeiro */
}
```

Exemplo: Verificação de Maioridade

```
#include <stdio.h>

int main() {
int idade;

printf("Informe a idade da pessoa:  ");
scanf("%d", &idade);

if (idade >= 18) {
printf("Pessoa maior de idade.\n");
}

return 0;
}
```

B) Seleção Composta (`if ... else`)

Executa o primeiro bloco se a condição for verdadeira e o bloco `else` se for falsa.

```
if (condicao) {
/* bloco verdadeiro */
} else {
/* bloco falso */
}
```

Exemplo: Média Escolar

Elabore um programa em C que leia duas notas, calcule a média e informe se o aluno está **Aprovado** ($Média \geq 8.0$) ou **Reprovado**.

```
#include <stdio.h>

int main() {
float nota1, nota2, media;

printf("Informe a primeira nota:  ");
scanf("%f", &nota1);

printf("Informe a segunda nota:  ");
scanf("%f", &nota2);

media = (nota1 + nota2) / 2.0;

if (media >= 8.0) {
printf("Aluno APROVADO com média:  %.2f\n", media);
} else {
printf("Aluno REPROVADO com média:  %.2f\n", media);
}

return 0;
}
```

5.2 2. Estruturas de Repetição (Laços / Loops)

Em vez de duplicar linhas de código para processar vários dados, utilizamos os laços de repetição nativos da linguagem C.

A) Repetição com Teste no Início (`while`)

Avalia a condição **antes** de executar o bloco. O laço continua rodando enquanto a condição for verdadeira.

```
while (condicao) {
/* comandos */
/* atualização da variável de controle */
}
```

Exemplo: Média da Turma de 10 Alunos

```
#include <stdio.h>

int main() {
char nome[50];
float nota1, nota2, media, somaNotas = 0.0;
int cont = 1; // Variável de controle

while (cont <= 10) {
printf("\n--- Aluno n%d ---\n", cont);
printf("Nome:  ");
scanf("%s", nome);

printf("1ª Nota:  "); scanf("%f", &nota1);
printf("2ª Nota:  "); scanf("%f", &nota2);

media = (nota1 + nota2) / 2.0;

if (media >= 9.0) {
printf("%s, média %.1f - APROVADO!\n", nome, media);
} else {
printf("%s, média %.1f - REPROVADO!\n", nome, media);
}

somaNotas = somaNotas + media; // Acumulador
cont = cont + 1;                // Passo essencial: incremento
}

printf("\nMédia geral do grupo:  %.2f\n", somaNotas / 10.0);
return 0;
}
```

Destaques em C

- **Incremento:** O comando `cont = cont + 1;` também pode ser escrito na forma simplificada de C: `cont++;`
- **Múltiplas leituras:** O `scanf("%s", nome)` lê strings simples sem espaço. Para ler nomes compostos, usa-se `fgets(nome, 50, stdin)`.

B) Repetição Determinada (`for`)

É a estrutura ideal quando sabemos o número exato de repetições. Em C, o `for` reúne a **inicialização**, a **condição** e o **incremento** em uma só linha.

```
for (inicializacao; condicao; incremento) {
/* comandos */
}
```

Exemplo: Listar Números Ímpares entre 120 e 180

```
#include <stdio.h>

int main() {
int cont;

for (cont = 120; cont <= 180; cont++) {
if (cont % 2 != 0) { // Em C, % calcula o resto da divisão
printf("%d é ímpar!\n", cont);
}
}

return 0;
}
```

C) Repetição com Teste no Final (`do ... while`)

Garante que o bloco seja executado **pelo menos uma vez** antes de testar a condição no final. Permanece em loop **enquanto a condição for verdadeira**.

```
do {
/* comandos */
} while (condicao);
```

Exemplo: Soma e Média de 30 Números

```
#include <stdio.h>

int main() {
int cont = 1, n, soma = 0, contImpar = 0;
float media;

do {
printf("Digite o %dº número:  ", cont);
scanf("%d", &n);

soma = soma + n;

if (n % 2 != 0) {
contImpar++;
}

cont++;
} while (cont <= 30);

media = (float)soma / 30.0;

printf("\nSoma total:  %d\n", soma);
printf("Quantidade de ímpares:  %d\n", contImpar);
printf("Média dos 30 números:  %.2f\n", media);

return 0;
}
```

6 Capítulo 6: Exercícios de Fixação

Agora é hora de praticar! Abaixo estão 7 problemas propostos em ordem crescente de dificuldade. Todos os exercícios utilizam os conceitos estudados até aqui: variáveis, atribuição, operadores, condicionais (`if/else`) e laços de repetição (`while/for/do-while`).

Dica para Resolução

Para cada problema, identifique antes de codificar:

1. Quais são os **dados de entrada** (o que o usuário deve digitar pelo `scanf`).
2. Qual é o **processamento** necessário (fórmulas e estruturas condicionais/repetição).
3. Qual é o **resultado de saída** esperado (o que exibir com `printf`).

Exercício 1: Área e Perímetro do Retângulo (Básico)

Escreva um programa em C que leia a largura e a altura de um retângulo. O programa deve calcular e exibir a área e o perímetro do retângulo.

```
Fórmulas:
- Área = largura * altura
- Perímetro = 2 * (largura + altura)
```

Exercício 2: Conversão de Temperatura (Básico)

Escreva um programa em C que leia uma temperatura em graus Celsius (C) e a converta para Fahrenheit (F). Ao final, exiba o resultado formatado.

```
Fórmula:
- F = (C * 1.8) + 32
```

Exercício 3: Cálculo da Hipotenusa (Fácil)

Um triângulo retângulo possui dois catetos a e b. Escreva um programa em C que receba o valor dos dois catetos e calcule a medida da hipotenusa (h).

```
Fórmula:
- h = sqrt(a*a + b*b)
Nota:  Para usar a função sqrt() (raiz quadrada), inclua a biblioteca
<math.h> no topo do programa.
```

Exercício 4: Classificação de Triângulos (Médio)

Escreva um programa em C que leia os três lados de um triângulo (A, B e C) e verifique a sua classificação:

- Equilátero: Todos os 3 lados são iguais.
- Isósceles: Apenas 2 lados são iguais.
- Escaleno: Todos os 3 lados são diferentes.

Exercício 5: Cálculo de Consumo de Combustível (Médio)

Um motorista deseja saber o desempenho de seu veículo em uma viagem. Faça um programa em C que leia a distância percorrida (em km) e a quantidade de combustível gasta (em litros). O programa deve calcular a média de consumo em km/l e classificar o veículo:

- Consumo menor que $8.0 km/l$: "Veículo Ineficiente
- Consumo entre 8.0 e $12.0 km/l$: "Consumo Normal"
- Consumo maior que $12.0 km/l$: "Econômico"

```
Fórmula:
- Consumo Médio = distância / combustível
```

Exercício 6: Somatório e Médias de Números Pares (Desafio Intermediário)

Escreva um programa em C que leia números inteiros positivos fornecidos pelo usuário até que o usuário digite o número `0` (que serve como chave de parada e não deve ser computado). Ao final, o programa deve exibir:

a) A quantidade de números pares digitados.

b) A média aritmética apenas dos números pares digitados.

```
Dica:
- Lembre-se de verificar se o número é par utilizando o operador resto (n
% 2 == 0).
- Cuidado com a divisão por zero ao calcular a média caso nenhum número
par seja digitado!
```

Exercício 7: Cálculo de Juros Compostos ao Longo do Tempo (Desafio Avançado)

Um investidor deseja simular o crescimento de um capital inicial ao longo de vários meses com uma taxa de juros fixa mensal.

Escreva um programa em C que leia:

- O capital inicial (P).
- A taxa de juros mensal em porcentagem (i). Exemplo: 1.5 para 1.5%.
- O total de meses (N).

O programa deve utilizar uma estrutura de repetição para simular mês a mês a evolução do montante, exibindo o saldo acumulado ao final de cada mês. Ao término do laço, exiba o rendimento total em reais.

```
Fórmula de atualização mensal:
- Novo Montante = Montante Anterior + (Montante Anterior * (taxa / 100))
```

7 Capítulo 7: Funções e Procedimentos

Conforme nossos programas crescem, escrever todo o código dentro da função principal `main()` torna o programa difícil de ler, testar e manter. Para resolver isso, utilizamos a modularização: a técnica de dividir um problema grande em pequenos blocos reutilizáveis.

Em C, esses blocos são chamados de Funções (quando retornam um valor) e Procedimentos (quando realizam uma ação sem retornar valor).

7.1 1. O que é um Procedimento? (`void`)

Um **procedimento** é uma rotina que executa um conjunto de instruções (como exibir mensagens na tela), mas **não devolve nenhum valor** para quem o chamou. Em C, indicamos isso usando a palavra-chave `void` no tipo de retorno.

```
void nome_do_procedimento() {
/* comandos que serão executados */
}
```

Exemplo: Procedimento de Boas-Vindas

```
#include <stdio.h>

// Definição do procedimento
void exibirCabecalho() {
printf("=====================================\n");
printf(" SISTEMA ACADÊMICO DE NOTAS \n");
printf("=====================================\n");
}

int main() {
// Chamada do procedimento
exibirCabecalho();
printf("Programa iniciado com sucesso.\n");

return 0;
}
```

7.2 2. O que é uma Função?

Uma função recebe dados de entrada (chamados de **parâmetros**), realiza um processamento e obrigatoriamente retorna um resultado do tipo especificado (`int`, `float`, etc.) usando o comando `return`.

```
tipo_de_retorno nome_da_funcao(tipo param1, tipo param2) {
/* processamento */
return resultado;
}
```

Exemplo 1: Função Somar

```
#include <stdio.h>

// Função que recebe dois inteiros e retorna a soma deles
int somar(int a, int b) {
return a + b;
}

int main() {
int resultado = somar(5, 3); // Chama a função e guarda o retorno

printf("A soma é:  %d\n", resultado);
return 0;
}
```

Exemplo 2: Função Lógica (Verificar Aprovação)

Em C clássico, representamos `true` (verdadeiro) como `1` e `false` (falso) como `0` usando o tipo `int` (ou usando a biblioteca `<stdbool.h>`).

```
#include <stdio.h>
#include <stdbool.h> // Permite usar o tipo bool (true/false)

// Função que retorna true se a média for >= 7.0 e false caso
contrário
bool estaAprovado(float media) {
if (media >= 7.0) {
return true;
} else {
return false;
}
}

int main() {
float mediaAluno = 8.5;

if (estaAprovado(mediaAluno)) {
printf("Situação:  APROVADO!\n");
} else {
printf("Situação:  REPROVADO!\n");
}

return 0;
}
```

7.3 3. Exercícios Propostos

Exercício 1: Procedimento de Alerta

Escreva um procedimento em C chamado `mensagemMenu()` que não receba parâmetros e apenas exiba na tela um menu formatado com 3 opções para o usuário (1- Cadastrar, 2- Listar, 3- Sair). No `main()`, faça apenas a chamada desse procedimento.

Exercício 2: Função de Validação de Aprovação

Crie um programa em C contendo:

1. Uma função chamada `verificarAprovacao` que receba duas notas (`float`), calcule a média simples e retorne `true` se a média for maior ou igual a 7.0, e `false` caso contrário.

2. Na função `main()`, solicite as duas notas ao usuário via `scanf`, chame a função criada e exiba a mensagem `"Aluno Aprovado"` ou `"Aluno Reprovado"` com base no retorno da função.